SIMATS ENGINEERING
ASSESSMENT TOOL 1-CO3
(Analytical Problem Solving)

COURSE CODE: ITA04
COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

QUESTIONS: 20 **Total Marks: 100**

ANNEXURE A

DESIGN TASK QUESTIONS

Code of Conduct:

I Rosenivitha J (192424009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DESIGN TASK QUESTIONS

1. Data Frames – Creation and Structure

A college wants to maintain student records containing Student ID, Name, Department, Marks, and Attendance. Analyze the suitability of a data frame for storing these records. Write an R program to create the data frame, display its structure, and identify the students who scored above 80 marks.

CODE

```
students <- data.frame(
  Student_ID = c(101, 102, 103, 104),
  Name = c("Arun", "Priya", "Kavi", "Ravi"),
  Department = c("IT", "CSE", "IT", "ECE"),
  Marks = c(85, 72, 91, 68),
  Attendance = c(90, 82, 95, 70))
print(students)
cat("\nStudents scoring above 80:\n")
print(students[students$Marks > 80, ])
```

OUTPUT

```
> print(students)
  Student_ID  Name Department Marks Attendance
1        101  Arun         IT    85         90
2        102 Priya        CSE    72         82
3        103  Kavi         IT    91         95
4        104  Ravi        ECE    68         70
> cat("\nStudents scoring above 80:\n")

Students scoring above 80:
> print(students[students$Marks > 80, ])
  Student_ID Name Department Marks Attendance
1        101 Arun         IT    85         90
3        103 Kavi         IT    91         95
```

2. Making Data Frames

A company maintains employee details, including Employee ID, Name, Age, Salary, and Department. Develop an R program to create a data frame using vectors. Analyze how R handles columns with different data types and explain the importance of maintaining equal-length vectors.

CODE

```
employee_id <- c(1, 2, 3)

name <- c("Arun", "Priya", "Ravi")

age <- c(22, 24, 23)

salary <- c(25000, 30000, 28000)

department <- c("IT", "HR", "CSE")

employees <- data.frame(

  Employee_ID = employee_id,

  Name = name,

  Age = age,

  Salary = salary,

  Department = department )

print(employees)

str(employees)
```

OUTPUT

```
> print(employees)
  Employee_ID  Name Age Salary Department
1           1  Arun  22  25000         IT
2           2 Priya  24  30000         HR
3           3  Ravi  23  28000        CSE
> str(employees)
'data.frame':   3 obs. of  5 variables:
 $ Employee_ID: num  1 2 3
 $ Name       : chr  "Arun" "Priya" "Ravi"
 $ Age        : num  22 24 23
 $ Salary     : num  25000 30000 28000
 $ Department : chr  "IT" "HR" "CSE"
```

3. Working with Data Frames

Consider a data frame containing student names, marks in three subjects, and attendance percentages. Write an R program to calculate each student's total and average marks, identify students who passed all subjects, and display students with attendance above 75%. Analyze how row and column operations simplify data processing.

CODE

```
students <- data.frame(
  Name = c("Arun", "Priya", "Ravi"),
  Maths = c(80, 65, 35),
  Statistics = c(75, 70, 40),
  Programming = c(90, 80, 30),
  Attendance = c(90, 85, 70) )
students$Total <- rowSums(students[, 2:4])
students$Average <- rowMeans(students[, 2:4])
cat("Student totals and averages:\n")
print(students)
cat("\nPassed all subjects (marks >= 40):\n")
```

```
print(students[students$Maths >= 40 &
                students$Statistics >= 40 &
                students$Programming >= 40, "Name"])
cat("\nAttendance above 75%:\n")
print(students[students$Attendance > 75, c("Name", "Attendance")])
```

OUTPUT

```
> cat("Student totals and averages:\n")
Student totals and averages:
> print(students)
   Name Maths Statistics Programming Attendance Total  Average
1  Arun    80         75          90         90   245 81.66667
2 Priya    65         70          80         85   215 71.66667
3  Ravi    35         40          30         70   105 35.00000
>
> cat("\nPassed all subjects (marks >= 40):\n")

Passed all subjects (marks >= 40):
> print(students[students$Maths >= 40 &
+                  students$Statistics >= 40 &
+                  students$Programming >= 40, "Name"])
[1] "Arun"  "Priya"
>
> cat("\nAttendance above 75%:\n")

Attendance above 75%:
> print(students[students$Attendance > 75, c("Name", "Attendance")])
   Name Attendance
1  Arun         90
2 Priya         85
```

4. Adding, Modifying, and Deleting Data

A library maintains a data frame containing Book ID, Title, Author, and Price. Develop an R program to add a new book, modify the price of an existing book, and delete a discontinued book. Analyze how these operations affect the structure and contents of the data frame.

CODE

```
books <- data.frame(
  Book_ID=c(101,102,103),
  Title=c("R Programming","Statistics","Python Basics"),
  Author=c("Arun","Priya","Kumar"),
  Price=c(500,600,450)
```

```
)

books <- rbind(books,data.frame(Book_ID=104,Title="C
Programming",Author="Ravi",Price=550))

books$Price[books$Book_ID==102] <- 650

books <- books[books$Book_ID!=103,]

print(books)
```

OUTPUT

```
  Book_ID         Title Author Price
1     101 R Programming   Arun   500
2     102    Statistics  Priya   650
4     104 C Programming   Ravi   550
```

5. Data Frame Indexing and Filtering

A retail store stores Product ID, Product Name, Category, Price, and Stock Quantity in a data frame. Write an R program to extract selected columns, filter products priced above ₹1,000, and identify products with stock below 10 units. Analyze the differences between row indexing, column indexing, and logical filtering.

CODE

```
products <- data.frame(

  Product_ID = c(101,102,103,104),

  Product_Name = c("Keyboard","Monitor","Mouse","Printer"),

  Category = c("Accessories","Electronics","Accessories","Electronics"),

  Price = c(800,6500,450,4500),

  Stock = c(20,5,8,12)

)
```

print(products[,c("Product_Name","Price")])

print(products[products$Price > 1000,])

print(products[products$Stock < 10,])

OUTPUT

```
> print(products[,c("Product_Name","Price")])
  Product_Name Price
1     Keyboard   800
2      Monitor  6500
3        Mouse   450
4      Printer  4500
> print(products[products$Price > 1000,])
  Product_ID Product_Name    Category Price Stock
2        102      Monitor Electronics  6500     5
4        104      Printer Electronics  4500    12
> print(products[products$Stock < 10,])
  Product_ID Product_Name    Category Price Stock
2        102      Monitor Electronics  6500     5
3        103        Mouse Accessories   450     8
```

6. Data Reshaping – Wide and Long Formats

A college records the marks of students in Mathematics, Statistics, and Programming using a wide-format data frame. Develop an R program to reshape the data into a long format and explain how the reshaped data supports subject-wise analysis. Compare the advantages of wide and long data formats.

CODE

wide <- data.frame(

  Student=c("Arun","Priya","Ravi"),

  Mathematics=c(80,90,70),

  Statistics=c(75,85,65),

  Programming=c(88,92,72)

)

```
long <- reshape(wide,varying=c("Mathematics","Statistics","Programming"),v.names="Marks",timevar="Subject",times=c("Mathematics","Statistics","Programming"),direction="long")

rownames(long) <- NULL

print(long)
```

OUTPUT

```
> print(long)
  Student     Subject Marks id
1    Arun Mathematics    80  1
2   Priya Mathematics    90  2
3    Ravi Mathematics    70  3
4    Arun  Statistics    75  1
5   Priya  Statistics    85  2
6    Ravi  Statistics    65  3
7    Arun Programming    88  1
8   Priya Programming    92  2
9    Ravi Programming    72  3
```

7. Melting Data Using R

A company records quarterly sales for four products across Q1, Q2, Q3, and Q4. Write an R program using the melt() function from the reshape2 package to convert the sales data into a long-format data frame. Analyze how melting simplifies product-wise and quarter-wise sales comparisons.

CODE

```
library(reshape2)

sales <- data.frame(

  Product=c("A","B","C","D"),

  Q1=c(100,120,90,150),

  Q2=c(110,130,95,160),

  Q3=c(115,125,105,170),
```

```
  Q4=c(125,140,110,180)

)

sales_long <- melt(sales,id.vars="Product",variable.name="Quarter",value.name="Sales")

print(sales_long)
```

OUTPUT

	Product	Quarter	Sales
1	A	Q1	100
2	B	Q1	120
3	C	Q1	90
4	D	Q1	150
5	A	Q2	110
6	B	Q2	130
7	C	Q2	95
8	D	Q2	160
9	A	Q3	115
10	B	Q3	125
11	C	Q3	105
12	D	Q3	170
13	A	Q4	125
14	B	Q4	140
15	C	Q4	110
16	D	Q4	180

8. Casting Data Using R

Using a long-format data frame containing Product, Quarter, and Sales, develop an R program using dcast() to convert the data back into a wide format. Generate a table showing products as rows and quarters as columns. Analyze how casting helps summarize and compare sales data.

CODE

```
library(reshape2)

sales <- data.frame(

  Product=rep(c("A","B","C"),each=4),

  Quarter=rep(c("Q1","Q2","Q3","Q4"),times=3),
```

```
  Sales=c(100,110,120,130,80,90,95,105,140,145,150,160)

)

sales_wide <- dcast(sales,Product~Quarter,value.var="Sales")

print(sales_wide)
```

OUTPUT

```
> print(wide_data)
  Product   Q1   Q2   Q3   Q4
1       A 1000 1200 1500 1800
2       B 2000 2200 2500 2800
3       C 1500 1700 1900 2100
```

9. Melting and Casting – Analytical Comparison

A hospital maintains patient records containing Patient ID, Department, and consultation expenses for three months. Design an R program to reshape the data using melting and casting techniques. Analyze how these transformations help compare monthly expenses across departments and identify the department with the highest expenditure.

CODE

```
data <- data.frame(

  Patient_ID=c(1,2,3,4,1,2,3,4,1,2,3,4),

Department=c("Cardiology","Neurology","Cardiology","Orthopedics","Cardiology","Neurology","Cardiology","Orthopedics","Cardiology","Neurology","Cardiology","Orthopedics"),

Month=c("January","January","January","January","February","February","February","February","March","March","March","March"),

  Expense=c(500,400,300,450,550,420,350,500,600,450,400,520)

)
```

```
monthly <-
aggregate(data$Expense,by=list(Department=data$Department,Month=data$Month),FUN=s
um)

names(monthly)[3] <- "Expense"

print(monthly)
```

OUTPUT

```
> print(monthly)
   Department    Month Expense
1  Cardiology February     900
2   Neurology February     420
3 Orthopedics February     500
4  Cardiology  January     800
5   Neurology  January     400
6 Orthopedics  January     450
7  Cardiology    March    1000
8   Neurology    March     450
9 Orthopedics    March     520
```

10. Merging Data Frames

Two data frames contain student details: one stores Student ID, Name, and Department, while the other stores Student ID, Marks, and Attendance. Write an R program to merge the data frames using Student ID as the common key. Analyze the differences between inner joins, left joins, right joins, and full joins, and justify which join is appropriate when all student records must be retained.

CODE

```
details <-
data.frame(Student_ID=c(1,2,3),Name=c("Arun","Priya","Ravi"),Department=c("IT","CSE",
"ECE"))

marks <- data.frame(Student_ID=c(1,2,4),Marks=c(85,90,75),Attendance=c(90,95,80))

print(merge(details,marks,by="Student_ID"))

print(merge(details,marks,by="Student_ID",all.x=TRUE))
```

print(merge(details,marks,by="Student_ID",all.y=TRUE))

print(merge(details,marks,by="Student_ID",all=TRUE))

OUTPUT

```
> print(merge(details,marks,by="Student_ID"))
  Student_ID  Name Department Marks Attendance
1          1  Arun         IT    85         90
2          2 Priya        CSE    90         95
> print(merge(details,marks,by="Student_ID",all.x=TRUE))
  Student_ID  Name Department Marks Attendance
1          1  Arun         IT    85         90
2          2 Priya        CSE    90         95
3          3  Ravi        ECE    NA         NA
> print(merge(details,marks,by="Student_ID",all.y=TRUE))
  Student_ID  Name Department Marks Attendance
1          1  Arun         IT    85         90
2          2 Priya        CSE    90         95
3          4  <NA>       <NA>    75         80
> print(merge(details,marks,by="Student_ID",all=TRUE))
  Student_ID  Name Department Marks Attendance
1          1  Arun         IT    85         90
2          2 Priya        CSE    90         95
3          3  Ravi        ECE    NA         NA
4          4  <NA>       <NA>    75         80
```

11. Merging Data with Missing Records

A company maintains separate data frames for employee details and monthly performance ratings. Some employees have no performance records, while some ratings refer to employees absent from the employee database. Develop an R program to merge these data frames using appropriate join operations. Analyze how unmatched records and missing values affect the final data set.

CODE

employees <- data.frame(Employee_ID=c(1,2,3),Name=c("Anu","Bala","Charan"))

ratings <- data.frame(Employee_ID=c(2,3,4),Rating=c(4,5,3))

print(merge(employees,ratings,by="Employee_ID",all.x=TRUE))

print(merge(employees,ratings,by="Employee_ID",all=TRUE))

OUTPUT

```
  Employee_ID   Name Rating
1           1    Anu     NA
2           2   Bala      4
3           3 Charan      5
> print(merge(employees,ratings,by="Employee_ID",a
  Employee_ID   Name Rating
1           1    Anu     NA
2           2   Bala      4
3           3 Charan      5
4           4   <NA>      3
```

12. Reading Data from Files

A researcher receives student performance data in a CSV file. Develop an R program to import the file using read.csv(), inspect its structure, identify missing values, and filter students whose average marks exceed 75. Analyze the importance of file paths, column names, data types, and missing-value handling during data import.

CODE

```
students <-
data.frame(Name=c("Arun","Priya","Ravi"),Maths=c(80,90,NA),Statistics=c(75,85,70),Progr
amming=c(90,88,80))

write.csv(students,"students.csv",row.names=FALSE)

data <- read.csv("students.csv",na.strings=c("","NA"))

str(data)

print(colSums(is.na(data)))

data$Average <- rowMeans(data[,c("Maths","Statistics","Programming")],na.rm=FALSE)

print(data[!is.na(data$Average)&data$Average>75,])
```

OUTPUT

```
> str(data)
'data.frame':   3 obs. of  4 variables:
 $ Name       : chr  "Arun" "Priya" "Ravi"
 $ Maths      : int  80 90 NA
 $ Statistics : int  75 85 70
 $ Programming: int  90 88 80
> print(colSums(is.na(data)))
        Name        Maths   Statistics Programming
           0            1            0            0
> data$Average <- rowMeans(data[,c("Maths","Statistics","Programming")],na.rm=FALSE)
> print(data[!is.na(data$Average)&data$Average>75,])
   Name Maths Statistics Programming  Average
1  Arun    80         75          90 81.66667
2 Priya    90         85          88 87.66667
```

13. Writing Data to Files

A sales analyst processes a data frame containing monthly revenue, expenses, and profit. Write an R program to calculate profit, select profitable months, and export the results using write.csv(). Analyze how exporting selected records supports reporting and data sharing, and explain the purpose of the row.names argument.

CODE

```
sales <- data.frame(Month=c("Jan","Feb","Mar","Apr"),Revenue=c(50000,60000,45000,70000),Expenses=c(40000,65000,30000,50000))

sales$Profit <- sales$Revenue-sales$Expenses

profitable <- sales[sales$Profit>0,]

print(profitable)

write.csv(profitable,"profitable_months.csv",row.names=FALSE)
```

OUTPUT

```
> print(profitable)
  Month Revenue Expenses Profit
1   Jan   50000    40000  10000
3   Mar   45000    30000  15000
4   Apr   70000    50000  20000
```

14. Editing and Updating File Data

A school stores student details in a CSV file. The administrator needs to correct student names, update marks, add new student records, and remove withdrawn students. Develop an R program to read the file, perform these modifications, and save the updated data to a new CSV file. Analyze the advantages of creating a separate output file instead of directly overwriting the original file.

CODE

```
school <- data.frame(Student_ID=c(1,2,3),Name=c("Arun","Priya","Ravi"),Marks=c(78,85,60),Status=c("Active","Active","Withdrawn"))

write.csv(school,"school.csv",row.names=FALSE)

data <- read.csv("school.csv")

data$Name[data$Student_ID==1] <- "Arun Kumar"

data$Marks[data$Student_ID==2] <- 90

new_student <- data.frame(Student_ID=4,Name="Meena",Marks=88,Status="Active")

data <- rbind(data,new_student)

data <- data[data$Status!="Withdrawn",]

write.csv(data,"school_updated.csv",row.names=FALSE)

print(data)
```

OUTPUT

```
> print(data)
  Student_ID       Name Marks Status
1          1 Arun Kumar    78 Active
2          2      Priya    90 Active
4          4      Meena    88 Active
```

15. Integrated Data Frame Analysis

An e-commerce company maintains two CSV files: products.csv, containing Product ID, Product Name, and Category, and sales.csv, containing Product ID, Month, and Sales Amount.

CODE

```
products <- data.frame(

  Product_ID = c(101,102,103),
```

```
  Product_Name = c("Keyboard","Monitor","Mouse"),
  Category = c("Accessories","Electronics","Accessories")
)
sales <- data.frame(
  Product_ID = c(101,101,102,103,103),
  Month = c("Jan","Feb","Jan","Jan","Feb"),
  Sales_Amount = c(5000,6000,15000,2000,2500)
)
write.csv(products,"products.csv",row.names=FALSE)
write.csv(sales,"sales.csv",row.names=FALSE)
products_data <- read.csv("products.csv")
sales_data <- read.csv("sales.csv")
combined <- merge(products_data,sales_data,by="Product_ID",all=TRUE)
print(combined)
totals <- aggregate(
  Sales_Amount ~ Product_ID + Product_Name + Category,
  data = combined,
  FUN = base::sum,
  na.rm = TRUE
)
print(totals)
```

OUTPUT

```
> print(combined)
  Product_ID Product_Name    Category Month Sales_Amount
1        101     Keyboard Accessories   Jan         5000
2        101     Keyboard Accessories   Feb         6000
3        102      Monitor Electronics   Jan        15000
4        103        Mouse Accessories   Jan         2000
5        103        Mouse Accessories   Feb         2500
>
> totals <- aggregate(
+   Sales_Amount ~ Product_ID + Product_Name + Category,
+   data = combined,
+   FUN = base::sum,
+   na.rm = TRUE
+ )
>
> print(totals)
  Product_ID Product_Name    Category Sales_Amount
1        101     Keyboard Accessories        11000
2        103        Mouse Accessories         4500
3        102      Monitor Electronics        15000
```

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Logic & Vectorisation	30%	**(28–30 Points)** • Zero iterative loops where native vectorised operators (&, |, %%, ifelse) apply. • Code handles complete vectors in single computational cycles.	**(23–27 Points)** • Accurate control setups.	**(15–22 Points)** • Heavy reliance on nested loops (for, while) for basic transformations. • Calculation code scales poorly and experiences significant execution delay.	**(0–14 Points)** • Confuses vector and scalar logic tracks. • Triggers frequent length mismatch errors or script termination crashes.

Argument & Flow Integrity	20%	**(19–20 Points)** • Complete use of dynamic lazy evaluation properties for defaults. • Integrates clean early-exit guard clauses to save memory cycles.	**(15–18 Points)** • Clear parameter layout.	**(10–14 Points)** • Missing critical default parameters. • Logic fails if a user leaves out an optional calculation parameter during runtime.	**(0–9 Points)** • Broken function signatures. • Erroneously uses internal assignment operators (<-) to specify signature defaults.
Object Architecture & Returns	20%	**(19–20 Points)** • Combines multiple metrics groups into a structured named list. • Completely relies on clean, implicit terminal line returns.	**(15–18 Points)** • Returns a single clean data structure (matrix/data frame) properly.	**(10–14 Points)** • Unpredictable return types across different internal calculation branches. • Output formats shift between vectors and matrices.	**(0–9 Points)** • Breaks isolated scope mechanics. • Updates external values using global modifications (<<-) instead of functional returns.
Edge Case Handling	15%	**(14–15 Points)** • Explicit validation blocks intercept empty inputs (numeric(0)) or missing data structures (NA, NaN, NULL) immediately.	**(11–13 Points)** • Code processes standard inputs cleanly. • Filters simple missing elements, but	**(8–10 Points)** • System crashes when an unexpected NA value is encountered inside strict logical comparisons.	**(0–7 Points)** • Complete lack of diagnostic checks. • Empty or missing data structures trigger fatal errors or

			unmapped structures occasionally leak into operations.		infinite loops immediately.
Recursion & Memory Strategy	15%	**(14–15 Points)** • Implements rigid mathematical base cases. • Zero copy-on-write memory overhead achieved via pre-allocation or vectorised setups.	**(11–13 Points)** • Restricts recursive tasks to safe internal depth limits. • Keeps basic iteration boundaries accurate.	**(8–10 Points)** • Recursion paths lack solid boundary conditions. • Threatens system stack safety if evaluated beyond small data sets.	**(0–7 Points)** • Infinite recursion due to a missing base case block. • Triggers immediate stack overflow system crashes.